# BETTERCASH

## Contrato da API · Referência Técnica

### Sistema de Gestão e Controle Financeiro com IA

**Projeto:** BetterCash — Mobile II
**Aluno:** Filipe Batista Ferreira
**Arquitetura:** MVVM + Factory Pattern
**Stack:** Flutter / Dart · API REST Mock
**Versão do Contrato:** 1.0 · Outubro/2026

Este documento define o contrato oficial da API do BetterCash, os padrões de pacote do projeto Flutter e a arquitetura MVVM + Factory que deve ser seguida em todos os componentes e telas.

# 1. Visão Geral do Projeto

BetterCash é um sistema de gestão e controle de finanças pessoais. O usuário registra receitas e despesas ao longo do mês. Em seguida, o sistema — com auxílio de Inteligência Artificial — analisa o comportamento financeiro e sugere formas de fazer o dinheiro render, impulsionando a saúde financeira do usuário.

**Funcionalidades obrigatórias:**

- Gerenciar usuários (cadastro, autenticação, perfil)
- Gerenciar transações (receitas, despesas e transferências)
- Analisar dados financeiros com IA (insights e recomendações)
- Gerar Dashboards com resumos e gráficos
- Exportar Dashboard com gráfico (PDF/PNG)
- Importar Extrato Bancário (CSV)
- Suporte a QR Code (compartilhamento / Pix — futuro)

## 1.1 Design System (Figma)

**UI Reference:** Free Finance UI Kit — https://www.figma.com/design/DHXVVUu3XT6Zr4BUN2isBd/Free-Finance-UI-Kit--Community---Copy-

**Design System:** https://www.figma.com/design/rUdlsvFRq93lD2hKOrfTqL/Design-System--Community---Copy-

> *Todo token de cor, espaçamento e tipografia deve ser extraído desses arquivos e centralizado em* `lib/common/theme/app_theme.dart`.

# 2. Arquitetura e Padrões de Pacote

O projeto adota **MVVM por componente** + **Factory como fábrica única**. Cada componente de UI possui três arquivos: ViewModel (estado + regras), Component (widget puro) e Factory (ponto único de criação). Telas consomem as Factories.

## 2.1 Diagrama da Arquitetura

Main → Screens (usam Factories) → Factory (cria componentes) → ViewModel (estado e regras) notifica → Component (Widget puro / UI). O módulo **Common** (Theme, Routes, BaseViewModel) é compartilhado por toda a aplicação.

## 2.2 Estrutura de Pastas Oficial

```
lib/
  common/                              ← recursos compartilhados
    theme/
      app_theme.dart                   (tokens: cor, espaçamento, tipografia)
    routes/
      app_routes.dart                  (gerenciador de rotas central)
    viewmodel/
      base_view_model.dart             (classe base com ChangeNotifier)
    models/                            (modelos de domínio compartilhados)
      user.dart
      transaction.dart
      summary.dart
      ai_insight.dart
    services/                          (clientes HTTP / repositórios)
      api_client.dart
      auth_service.dart
      transaction_service.dart
      ai_service.dart
      dashboard_service.dart
  components/
    action_button/
      action_button_view_model.dart    ← estado + regra (loading, enabled)
      action_button_component.dart     ← widget puro, só desenha
```

```
    action_button_factory.dart       ← ponto único de criação
  loading/                           (mesma estrutura)
  tab_bar/                           (mesma estrutura)
  list_items/                        (mesma estrutura)
  transaction_card/                  (mesmo padrão)
  chart_card/                        (mesmo padrão)
screens/
  login_screen.dart
  home_screen.dart
  transactions_screen.dart
  dashboard_screen.dart
  profile_screen.dart
  sample_screen.dart                 ← catálogo de componentes
main.dart
```

## 2.3 Regras do Padrão MVVM + Factory

• **ViewModel**: contém estado, regras de negócio da UI e notifica listeners (ChangeNotifier). Nunca importa Flutter Material diretamente além do necessário.

• **Component**: Widget puro (Stateless preferencialmente). Recebe o ViewModel e apenas desenha. Sem lógica de negócio.

• **Factory**: único ponto de criação do par ViewModel + Component. Screens e outros componentes só chamam a Factory.

• **Common**: Theme, Routes e BaseViewModel. Nenhuma tela ou componente deve duplicar tokens ou lógica de navegação.

• **Services**: camada de acesso à API. ViewModels consomem Services, nunca fazem HTTP direto.

• **Models**: classes Dart imutáveis (ou freezed) que espelham os JSON da API.

# 3. Regras Gerais da API

Todos os endpoints do servidor mock, com verbo, formato da requisição e resposta esperada. O resumo das regras está neste contrato. Exemplos capturados do servidor rodando (versão 1.0.0).

| Item | Regra |
|---|---|
| Base URL | http://localhost:8080. No emulador Android: http://10.0.2.2:8080 |
| Formato | Corpo e resposta em JSON, Content-Type: application/json; charset=utf-8 |
| Autenticação | Rotas protegidas exigem Authorization: Bearer (retornado no login) |
| Dados por sessão | Cada login recebe cópia própria do seed. O que uma sessão altera, outra não vê |
| Datas | ISO 8601 em UTC (ex: 2026-09-06T14:32:00Z) |
| Valores monetários | number com duas casas, sempre positivo. O type define se entra ou sai |
| Campos opcionais | Vêm sempre; null quando vazios |
| Latência | 300 a 800 ms por padrão (LATENCY_MIN_MS / LATENCY_MAX_MS) |
| CORS | Qualquer origem aceita; OPTIONS responde 204 |
| Credenciais de teste | ana@bettercash.dev / 123456 e bruno@bettercash.dev / 123456 |

# 4. Visão Geral dos Endpoints

| Verbo | Rota | Auth | Sucesso |
|---|---|---|---|
| GET | /health | não | 200 |
| POST | /auth/login | não | 200 |
| POST | /auth/logout | sim | 204 |
| GET | /me | sim | 200 |
| PUT | /me | sim | 200 |
| GET | /transactions | sim | 200 |
| GET | /transactions/:id | sim | 200 |
| POST | /transactions | sim | 201 |
| PUT | /transactions/:id | sim | 200 |
| DELETE | /transactions/:id | sim | 204 |
| POST | /transactions/import | sim | 200 |
| GET | /summary | sim | 200 |
| GET | /dashboard | sim | 200 |
| GET | /dashboard/export | sim | 200 |
| POST | /ai/insights | sim | 200 |
| GET | /ai/insights/:id | sim | 200 |
| POST | /reset | sim | 204 |

# 5. Modelos de Dados

## 5.1 user

```
{
  "id": "usr_01",
  "name": "Ana Ribeiro",
  "email": "ana@bettercash.dev",
  "avatarUrl": null,
  "createdAt": "2024-03-11T09:00:00Z"
}
```

## 5.2 transaction

```
{
  "id": "tx_0042",
  "type": "expense",
  "amount": 89.90,
  "description": "Mercado",
  "category": "Alimentação",
  "date": "2026-09-06T14:32:00Z",
  "note": null,
  "account": "Nubank",
  "toAccount": null
}
```

| Campo | Tipo | Regra |
|---|---|---|
| type | string | income \| expense \| transfer |
| amount | number | maior que zero; arredondado a duas casas |
| description | string | não vazia |
| category | string? | texto vazio vira null |
| date | string | ISO 8601 válida; devolvida em UTC |
| note | string? | texto vazio vira null |
| account | string | não vazia |
| toAccount | string? | obrigatório se type = transfer; senão null |

*Nota: Transferência não entra no cálculo de saldo (income – expense).*

## 5.3 summary

```
{
  "month": "2026-09",
  "income": 9526.55,
  "expense": 5330.04,
  "balance": 4196.51
}
```

## 5.4 dashboard

```
{
  "month": "2026-09",
  "income": 9526.55,
  "expense": 5330.04,
  "balance": 4196.51,
  "byCategory": [
    { "category": "Alimentação", "amount": 1840.20, "percent": 34.5 },
    { "category": "Transporte",  "amount": 920.00,  "percent": 17.3 }
  ],
  "dailyBalance": [
    { "date": "2026-09-01", "balance": 1200.00 },
```

```
    { "date": "2026-09-02", "balance": 980.50 }
  ],
  "topExpenses": [ /* array de transaction */ ]
}
```

## 5.5 ai_insight

```
{
  "id": "ins_001",
  "month": "2026-09",
  "generatedAt": "2026-09-28T10:15:00Z",
  "summary": "Você gastou 18% a mais com Alimentação este mês.",
  "recommendations": [
    {
      "title": "Reduza delivery",
      "description": "Substituir 4 deliveries por refeições caseiras pode economizar ~R$ 320/mês.",
      "potentialSaving": 320.00,
      "priority": "high"
    }
  ],
  "score": 72
}
```

# 6. Formato de Erro

Toda falha usa o mesmo envelope. Em 422 o objeto ganha o campo `fields` com uma mensagem por campo inválido.

```
{
  "error": {
    "code": "validation",
    "message": "Invalid transaction",
    "fields": {
      "amount": "must be greater than zero",
      "description": "must not be empty"
    }
  }
}
```

| Status | Code | Quando |
|---|---|---|
| 400 | bad_request | corpo não é JSON válido |
| 401 | invalid_credentials | e-mail ou senha errados no login |
| 401 | unauthorized | token ausente, desconhecido ou já encerrado |
| 401 | token_expired | só com X-Fail: expired |
| 404 | not_found | recurso ou rota inexistente |
| 422 | validation | campos ou parâmetros inválidos |
| 500 | internal | erro inesperado ou X-Fail: 500 |

# 7. Endpoints Detalhados

## 7.1 Saúde — GET /health

Verifica se o servidor está no ar. Rota pública.

**Resposta 200:**

```
{ "status": "ok", "version": "1.0.0" }
```

## 7.2 Autenticação

**POST /auth/login** — Autentica e cria sessão com dados próprios. Pública.

Requisição:

```
POST /auth/login
Content-Type: application/json

{ "email": "ana@bettercash.dev", "password": "123456" }
```

Resposta 200:

```
{
  "token": "tok_0vu34w77pn3oquql7xjtqp5r",
  "user": {
    "id": "usr_01",
    "name": "Ana Ribeiro",
    "email": "ana@bettercash.dev",
    "avatarUrl": null,
    "createdAt": "2024-03-11T09:00:00Z"
  }
}
```

*Erros: 400 (JSON malformado), 401 (invalid_credentials), 422 (fields vazios).*

**POST /auth/logout** — Encerra a sessão. Token passa a retornar 401.

```
POST /auth/logout
Authorization: Bearer <token>

→ 204 No Content
```

## 7.3 Perfil

**GET /me** — Devolve o usuário da sessão.

**PUT /me** — Atualiza nome e avatarUrl. Demais campos não mudam.

```
PUT /me
Authorization: Bearer <token>
Content-Type: application/json

{ "name": "Ana R.", "avatarUrl": null }
```

*name: obrigatório, não vazio (trim). avatarUrl: string ou null (vazio → null).*

## 7.4 Lançamentos (Transações)

**GET /transactions** — Lista do mais recente para o mais antigo, com filtros e paginação.

```
GET /transactions?month=2026-09&type=expense&q=mercado&page=1&pageSize=20
Authorization: Bearer <token>
```

| Parâmetro | Obrig. | Padrão | Regra |
|---|---|---|---|
| month | não | todos | YYYY-MM |
| type | não | todos | income \| expense \| transfer |
| q | não | — | busca na description (case-insensitive) |
| page | não | 1 | inteiro ≥ 1 |
| pageSize | não | 20 | inteiro de 1 a 100 |

*Resposta 200: { items: [...], page, pageSize, total }. Página além do fim → items: [].*

**GET /transactions/:id** — Devolve um lançamento. 404 se não existir.

**POST /transactions** — Cria. Servidor gera o id (id no corpo é ignorado). Resposta 201.

**PUT /transactions/:id** — Substitui o lançamento inteiro. Id do path prevalece.

**DELETE /transactions/:id** — Remove. 204. 404 se já removido ou inexistente.

## 7.5 Importação de Extrato (CSV)

**POST /transactions/import** — Recebe um arquivo CSV (multipart/form-data) ou JSON com array de linhas. O servidor normaliza, valida e cria as transações. Retorna resumo da importação.

```
POST /transactions/import
Authorization: Bearer <token>
Content-Type: multipart/form-data

file: extrato.csv

// ou JSON:
{
  "rows": [
    { "date": "2026-09-05", "description": "PIX Recebido", "amount": 1500.00, "type": "income" },
    { "date": "2026-09-06", "description": "Mercado", "amount": 89.90, "type": "expense" }
  ]
}
```

Resposta 200:

```
{
  "imported": 42,
  "skipped": 3,
  "errors": [
    { "line": 7, "message": "amount must be greater than zero" }
  ]
}
```

## 7.6 Resumo e Dashboard

**GET /summary?month=2026-09** — Soma receitas e despesas do mês (transferências fora).

```
{ "month": "2026-09", "income": 9526.55, "expense": 5330.04, "balance": 4196.51 }
```

**GET /dashboard?month=2026-09** — Dados ricos para gráficos: byCategory, dailyBalance, topExpenses.

**GET /dashboard/export?month=2026-09&format;=pdf** — Retorna o arquivo (PDF ou PNG) do dashboard com gráficos. Content-Type: application/pdf ou image/png. Também aceita format=png.

### 7.7 Análise com IA

**POST /ai/insights** — Dispara análise do mês informado (ou mês atual). O servidor mock gera insights determinísticos baseados nas transações da sessão. Em produção seria chamada a um modelo de linguagem + regras de negócio.

```
POST /ai/insights
Authorization: Bearer <token>
Content-Type: application/json

{ "month": "2026-09" }
```

Resposta 200: objeto ai_insight completo.

**GET /ai/insights/:id** — Recupera um insight já gerado.

### 7.8 Utilitários

**POST /reset** — Devolve os dados da sessão ao estado do seed. O token continua válido. 204.

## 8. Cabeçalhos de Simulação

Valem em qualquer rota e são avaliados antes da autenticação. Úteis para testes de loading, erro e timeout no app Flutter.

| Cabeçalho | Efeito |
|---|---|
| X-Delay-Ms: 3000 | usa exatamente esta latência, em ms |
| X-Fail: 500 | 500 com {"error":{"code":"internal","message":"Simulated failure"}} |
| X-Fail: expired | 401 com {"error":{"code":"token_expired","message":"Token has expired"}} |
| X-Fail: timeout | segura a requisição por 60 s e responde 504 em texto puro |

*Com token válido, rota inexistente → 404 {"error":{"code":"not_found","message":"Route not found"}}.*

## 9. Mapeamento para o App Flutter (MVVM)

Cada endpoint deve ter um método correspondente no Service apropriado. O ViewModel da tela chama o Service, atualiza o estado e notifica o Component.

| Endpoint | Service | ViewModel típico |
|---|---|---|
| POST /auth/login | AuthService | LoginViewModel |
| GET/PUT /me | AuthService | ProfileViewModel |
| CRUD /transactions | TransactionService | TransactionsViewModel / TransactionFormViewModel |
| POST /transactions/import | TransactionService | ImportViewModel |
| GET /summary + /dashboard | DashboardService | DashboardViewModel |
| GET /dashboard/export | DashboardService | DashboardViewModel |
| POST /ai/insights | AiService | InsightsViewModel |

### 9.1 Fluxo recomendado em uma tela

1. Screen chama a Factory do componente principal (ex: TransactionListFactory.create()).
2. A Factory instancia o ViewModel e o Component, injetando o Service necessário.
3. No init do ViewModel, chama-se o Service (ex: transactionService.list(...)).
4. Service usa ApiClient (com token do AuthService) e retorna Models tipados.
5. ViewModel atualiza listas/flags (isLoading, error, items) e notifica.
6. Component escuta o ViewModel e reconstrói a UI de forma reativa.

# 10. Checklist de Entrega (Mobile II)

- ■ Tema extraído do Figma e centralizado em app_theme.dart (sem valores placeholder).
- ■ Estrutura de pastas idêntica à seção 2.2.
- ■ BaseViewModel implementado e herdado por todos os ViewModels.
- ■ ActionButton (primary / secondary / outline) validado contra o Figma.
- ■ Pelo menos um segundo componente obrigatório (TabBar ou ListItems) com Factory.
- ■ Telas de Login, Home/Dashboard, Lista de Transações e Perfil navegáveis.
- ■ Integração real com os endpoints de auth + transactions + summary (ou mock local fiel).
- ■ Tratamento de loading, erro e empty state em todos os ViewModels de tela.
- ■ README no repositório descrevendo o padrão MVVM + Factory e link do Figma.
- ■ Commit inicial + pushes organizados no GitHub.

---

*Documento gerado para o curso de Formação Mobile em Dart / Flutter — BetterCash · Mobile II. Este contrato serve como fonte única de verdade entre o app Flutter e o servidor mock.*